1. ***Parameterised queries:*** Instead of using SQL queries directly in your application, use parameterised queries. These have exactly the same results as normal SQL queries but the difference is that, here, you need to define the SQL code first and then pass the parameters to the query later. So, even if someone tries to attack by passing malicious data to the query, the query searches for the exact match of whatever is sent as input. For example, if someone tries to pass OR '1'='1' as the data, the query will look up the DB for a literal match of the data. So that when this text is sent as parameter to the query, it becomes something like -SELECT * FROM users WHERE name = '' OR '1' = '1'. Here is an example of how to write parameterised queries in PHP. Check out more about parameterised queries in your programming language manual.

```
/*Prepared statement, stage 1: prepare */
if (!($stmt = $mysqli->prepare("INSERT INTO test(id) VALUES
(?)"))) {
echo "Prepare failed: (" . $mysqli->errno . ") " . $mysqli-
>error;
}

/* Prepared statement, stage 2: bind and execute */
$id = 1;
if (!$stmt->bind_param("i", $id)) {
     echo "Binding parameters failed: (" . $stmt->errno . ")
" . $stmt->error;
}

if (!$stmt->execute()) {
    echo "Execute failed: (" . $stmt->errno . ") " . $stmt-
>error;
}
```

So, the next time you need to look up the database, use a parameterised query for it. But beware, this approach has a downside as well; in some cases, this can harm performance, because parameterised queries need server resources.

2. ***Session management implementation:*** You should always use the inbuilt session management feature, which is available out-of-the-box with your Web development framework. Not only does this save critical development time and cost, it is generally safer as well, since there are many people out there using and testing it. Another thing to take care of while implementing session management is to keep track of what method the application uses to send or receive the session ID. It may be a cookie, URL rewriting or a mixture of both, but you should generally limit it and accept the session ID only via the mechanism you chose in the first place.
3. ***Cookie management:*** Whenever you plan to use cookies, be aware that you are sending out data about the user/session, which can potentially be intercepted and misused. So, you need to be extra careful while handling cookies. Always add the two cookie attributes, *secure* and *HttpOnly*, since this ensures the cookie is always sent only via an Https connection and doesn't allow scripts to access the cookie. These two attributes will reduce the chances of cookies being intercepted. Other attributes like *domain* and *path* also should always be set to indicate to the browser where exactly the cookie should be sent, so that it reaches only the exact destination and not anywhere else. Last, but definitely not the least, the *expire* and *maximum age* attributes should be set in order to make the cookie non-persistent so that it is wiped off once the browser instance is closed.
4. ***Session expiry management:*** A time-out should be enforced over and above the idle time-out, so that if the users intend to stay longer, they should authenticate themselves again. This generates a new session ID and so, the hacker has less time to crack the session ID. Also, when the session is being invalidated, special care should be taken to clear the browser data and the cookie, if used. To invalidate a cookie, set the session ID to *empty* and the expiry date to a past date. Similarly, the server side should also close and invalidate the session by calling proper session handling methods, for example *session_destroy()/ unset()* in PHP.
5. ***XML denial of service prevention:*** Denial of service attacks attempt to flood the Web server with a large number of requests so that it eventually crashes. To protect your Web service from such attacks, be sure to optimise the configuration for maximum message throughput and limit the SOAP message size. Also, validate the requests against recursive or oversized payloads, since such payloads can generally be malicious.

This brings us to the end of this article on software security, but I must add that this is just the tip of the iceberg. With rapidly improving computing techniques, the frequency and intensity of attacks is increasing, and developers should create software that can face them. For this, I would suggest that all developers acquaint themselves with OWASP security guidelines. Feel free to write back with any queries or feedback you may have. END

By: Nitish Kumar Tiwari

The author is based in Bengaluru, India, and works as a software developer for a FOSS-based firm. In his free time, he likes to dabble with open source software and share his experiences. Other interests include movies and travelling. Contact him at *Nitish.tiwari@technoocube.in.*